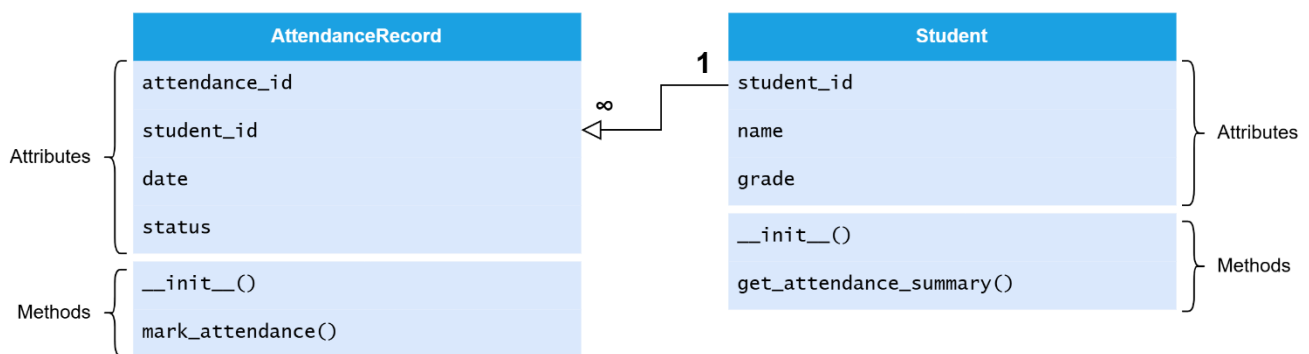


School Attendance Tracker

- **UI:** First screen summarizes the students with all of the fields in a datasheet format, user selects a record and clicks RECORD ATTENDANCE, which pulls them into a screen with student details, a date field, and the option to select present/not present and click submit.
- **Tables:** *Students* (student_id, name, grade), *Attendance* (attendance_id, student_id, date, status).
- **Functionality:** Track attendance and student information.

UML Diagrams for Two Python Classes



Relational Database Design – Two Linked Tables

Students Table

Field Name	Data Type	Description
student_id	Integer	Primary Key – Unique student ID
name	Text	Full name of student
grade	Integer	Student's grade in a class

Attendance Table

Field Name	Data Type	Description
attendance_id	Integer	Primary Key – Unique attendance record ID
student_id	Integer	Foreign Key – ID of the student (linked to Students table)
date	Text	Date of attendance (ex. "2025-04-21")
status	Text	"Present" or "Absent"

Explanation of Class ↔ Table Mapping

Between the two tables, there is a one-to-many relationship using the `student_id` field (one student can have many attendance records). I've uploaded the `attendance_tracker.db` file, which contains both tables with sample records, as well as the `create_tables_and_records.sql` file, which includes all the SQL statements used to create the database. The `Student` class maps directly to the `Students` table, where `student_id` is the primary key that uniquely identifies each student, and the class attributes (`student_id`, `name`, and `grade`) correspond directly to the table columns. The `AttendanceRecord` class maps to the `Attendance` table, where `attendance_id` is the primary key and `student_id` is a foreign key linking back to the `Students` table. This structure reflects the one-to-many relationship, allowing each student to have multiple attendance records associated with their ID.